

UNIVERSIDAD TECNICA DE MANABI

TECNOLOGIA DE LA INFORMACION Y COMUNICACION

NOMBRE: NEYDA MAGALY ZAMBRANO MACIAS

ASIGNATURA: DESARROLLO DE SISTEMAS INFORMATICOS

TEMA: ESTUDIO DE CASO Y SISTEMA DE CONTROL DE RSCATE Y ADOPCION ANIMAL.

DOCENTE: ING. GORGE ZAMBRANO

PERIODO: SEPTIEMBRE- ENERO 2027

ENLACE DEL GITHUB: https://github.com/NeydaMagaly/Desarrollo-de-Sistemas-informaticos-/tree/feature/analisis-requerimientos

ESTUDIO DE CASO Y SISTEMA DE CONTROL DE RSCATE Y ADOPCION ANIMAL.

1. Introducción

El cuidado y la protección de los animales son actividades que requieren organización, responsabilidad y una adecuada coordinación entre las personas que participan en los rescates. En un refugio animal, cada caso necesita un seguimiento desde el momento en que se reporta una situación de riesgo hasta que el animal recibe atención veterinaria, es adoptado o, cuando corresponde, regresa a su hábitat.

Sin embargo, cuando la información se maneja de manera manual o se encuentra dispersa en diferentes registros, resulta más difícil conocer el estado de cada caso, identificar a los responsables y mantener un historial actualizado.

Por eso, en este trabajo se propone el diseño de un Sistema de Control de Rescate y Adopción Animal, cuyo propósito es organizar la información de los animales rescatados y facilitar el trabajo de ciudadanos, voluntarios, veterinarios y administradores del refugio.

Para plantear una solución ordenada, se utilizarán diagramas UML, patrones de diseño y principios SOLID. También se empleará GitHub con el flujo de trabajo Gitflow para organizar los archivos, registrar los avances y mantener un historial de los cambios realizados durante el desarrollo del diseño.

2. Objetivos

2.1. Objetivo general

Diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal mediante diagramas UML, patrones de diseño y Gitflow, con la finalidad de organizar los procesos del refugio y proponer una solución mantenible, escalable y comprensible.

2.2. Objetivos específicos

Identificar los actores que intervienen en el sistema y definir sus principales necesidades.
Establecer los requerimientos funcionales y no funcionales relacionados con la gestión de rescates y adopciones.
Representar la estructura y el comportamiento del sistema mediante diagramas UML de casos de uso, clases y secuencia.

Aplicar los patrones Singleton, Observer, Factory Method y MVC, justificando su utilidad dentro de la solución.
Utilizar los principios SOLID para distribuir adecuadamente las responsabilidades de las clases.
Organizar los archivos del proyecto mediante GitHub y Gitflow, conservando evidencias verificables de las ramas y los cambios realizados.

3. Planteamiento del problema

Un refugio animal puede recibir reportes de perros, gatos y animales silvestres que necesitan ayuda. Cada situación requiere recopilar información, identificar la especie, registrar las condiciones en las que se encontró al animal y determinar quién se encargará de su atención.

Cuando no existe un sistema centralizado, pueden presentarse dificultades para consultar los expedientes, coordinar al personal, actualizar los estados de los casos y conocer qué animales se encuentran en tratamiento o están disponibles para adopción.

Ante esta necesidad, se plantea diseñar un sistema que permita registrar los rescates, asignar responsables, controlar las etapas de atención y mantener la información organizada. La solución también deberá facilitar las notificaciones sobre nuevos casos y permitir que el sistema pueda ampliarse en el futuro sin tener que modificar toda su estructura.

4. Justificación

La propuesta de este sistema nace de la necesidad de administrar de una manera más ordenada la información relacionada con el rescate y la adopción animal. Contar con expedientes digitales permitiría consultar los datos de cada animal, conocer su situación actual y reconocer a las personas responsables de su seguimiento.

Desde el punto de vista del desarrollo de software, esta actividad permite aplicar los conocimientos de diseño orientado a objetos mediante patrones que ayudan a distribuir las responsabilidades del sistema. Además, el uso de UML facilita la comprensión de su estructura antes de comenzar una implementación completa.

Por otra parte, GitHub y Gitflow permiten organizar el trabajo en ramas, registrar los avances y conservar evidencia de los cambios realizados. De esta manera, el proyecto no solo busca responder a una necesidad del refugio, sino también demostrar la aplicación de buenas prácticas de ingeniería de software.

5. Análisis del sistema

5.1 Descripción del sistema

El Sistema de Control de Rescate y Adopción Animal se plantea como una solución informática para mejorar la organización y el seguimiento de los animales que ingresan a un refugio. Su propósito es centralizar la información de cada rescate, facilitar la asignación de responsables y mantener un registro actualizado de las actividades realizadas durante la atención del animal.

El sistema permitirá registrar reportes realizados por los ciudadanos, identificar la especie del animal y generar un expediente con los datos correspondientes. También permitirá que los administradores asignen voluntarios o veterinarios, según las necesidades de cada caso.

Una vez registrado el expediente, se podrá actualizar su estado de acuerdo con el proceso de atención. Entre los estados propuestos se encuentran *Rescatado*, *En tratamiento*, *Disponible para adopción*, *Adoptado* y *Liberado*, cuando corresponda a un caso de fauna silvestre y se cumplan las condiciones necesarias.

Además, se propone incorporar un mecanismo de notificaciones para informar a los voluntarios suscritos sobre los nuevos rescates registrados. De esta manera, el sistema busca mejorar la coordinación del personal, reducir la dispersión de la información y facilitar la consulta del historial de cada expediente.

5.2. Identificación de actores

Los actores representan a las personas que interactúan con el sistema y realizan actividades específicas de acuerdo con sus responsabilidades.

Ciudadano o reportante

Es la persona que informa sobre un animal abandonado, herido o que necesita ayuda. Su función principal es proporcionar los datos iniciales del reporte, como la ubicación, la descripción de la situación y la información disponible sobre el animal. También podrá consultar el estado de su reporte, de acuerdo con los permisos establecidos.

Voluntario

Es la persona que colabora con las actividades de rescate y atención de los animales. Mediante el sistema podrá consultar los casos que le hayan sido asignados, revisar la

información necesaria y registrar las actualizaciones operativas que estén dentro de sus permisos. Asimismo, podrá recibir notificaciones sobre nuevos rescates cuando se encuentre suscrito al servicio correspondiente.

Veterinario

Es el profesional responsable de registrar la información relacionada con la salud de los animales atendidos. Entre sus funciones se encuentran documentar diagnósticos, registrar tratamientos y actualizar las observaciones médicas de los expedientes asignados. Su participación permite mantener un historial de atención veterinaria para cada animal.

Administrador del refugio

Es el encargado de supervisar las operaciones generales del sistema. Puede registrar los ingresos de animales rescatados, clasificar las especies, asignar voluntarios o veterinarios y controlar el seguimiento de los expedientes. También puede gestionar las solicitudes de adopción y consultar el historial de los casos para verificar su evolución.

5.3. Requerimientos funcionales

Los requerimientos funcionales describen las operaciones que el sistema debe permitir realizar para cumplir con las necesidades del refugio.

Registrar Reportes	El sistema permitirá registrar reportes de animales que necesiten rescate, incluyendo ubicación y descripción.
Clasificar Especies	Permitirá identificar si el animal es un perro, un gato o pertenece a la fauna silvestre.
Crear Expedientes	Generará un expediente para cada caso registrado, asociado al animal correspondiente.
Asignar responsables	Permitirá al administrador asignar voluntarios o veterinarios según las necesidades del caso.
Actualizar estados	Actualizar el estado del expediente conforme avance la atención.
Registrar tratamientos	Documentar diagnósticos, tratamientos y observaciones veterinarias.
Evitar notificaciones	Informar a los voluntarios suscritos sobre nuevos rescates.
Gestionar adopciones	Registrar y dar seguimiento a las solicitudes de adopción.

Consultar expedientes	Consultar los datos de los casos según los permisos de cada usuario.
Mantener historial	Registrar las actualizaciones importantes de cada expediente.

5.4. Requerimientos no funcionales

Los requerimientos no funcionales establecen las condiciones de calidad que deberá cumplir el sistema. No describen directamente una operación, sino las características que permitirán que la solución sea confiable, segura y fácil de mantener.

Escalabilidad	Permitir incorporar nuevas especies y funcionalidades sin modificar toda la arquitectura.
Mantenibilidad	Organizar el código en componentes con responsabilidades definidas.
Seguridad	Controlar el acceso mediante autenticación y permisos por rol.
Trazabilidad	Mantener un historial de cambios mediante Git y GitHub.
Fiabilidad	Validar los datos y controlar las transiciones entre estados.
Usabilidad	Presentar una interfaz clara para registrar y consultar información.
Integridad	Mantener relaciones coherentes entre los animales, los casos y los responsables.

5.5. Reglas de negocio

Para mantener la coherencia de la información, el sistema deberá considerar las siguientes reglas:

1. Cada caso de rescate deberá tener un identificador único que permita localizar su expediente.
2. Todo animal registrado deberá estar asociado con un caso de rescate.
3. La especie seleccionada determinará el tipo de objeto que se creará mediante la fábrica de expedientes.
4. La asignación de voluntarios y veterinarios deberá realizarse de acuerdo con sus funciones y permisos.
5. Los cambios de estado deberán registrarse para conservar el historial del expediente.
6. Los tratamientos veterinarios solo podrán ser registrados por usuarios autorizados para esa función.
7. La adopción deberá registrarse cuando se hayan cumplido las condiciones definidas por el refugio.

8. En los casos de fauna silvestre, la liberación deberá gestionarse conforme a la evaluación correspondiente y a los procedimientos aplicables.

Estas reglas permitirán establecer límites claros para las operaciones del sistema y ayudarán a prevenir inconsistencias durante la gestión de los casos.

5.6. Alcance del sistema

La propuesta comprende el registro de reportes, la creación de expedientes, la clasificación de especies, la asignación de responsables, el seguimiento de estados, el registro de tratamientos y la gestión de adopciones. También contempla notificaciones y un historial de las operaciones importantes.

El diseño se enfoca en la arquitectura lógica y en la representación del sistema mediante UML y patrones de diseño. Por tanto, las funcionalidades descritas constituyen el comportamiento esperado de la solución y deberán implementarse y probarse posteriormente para comprobar su funcionamiento.

La administración de una base de datos, la autenticación de usuarios y el envío real de notificaciones podrán incorporarse durante una fase posterior de desarrollo.

6. Diseño de la arquitectura

Arquitectura general propuesta

Se propone una arquitectura organizada mediante el patrón MVC, complementada con servicios de negocio y una capa de persistencia. Esta distribución permite separar la presentación, el procesamiento de solicitudes, las reglas del dominio y el almacenamiento de los datos.

La arquitectura se compone de los siguientes elementos:

- **Vista (View):** presenta los formularios para registrar rescates, consultar expedientes y gestionar adopciones.
- **Controlador (Controller):** recibe las solicitudes de los usuarios, valida los datos de entrada y coordina las operaciones.
- **Modelo (Model):** representa las entidades del sistema, como Animal, CasoRescate y Usuario.
- **Servicios de negocio:** coordinan el registro de los casos, la creación de animales y las notificaciones.

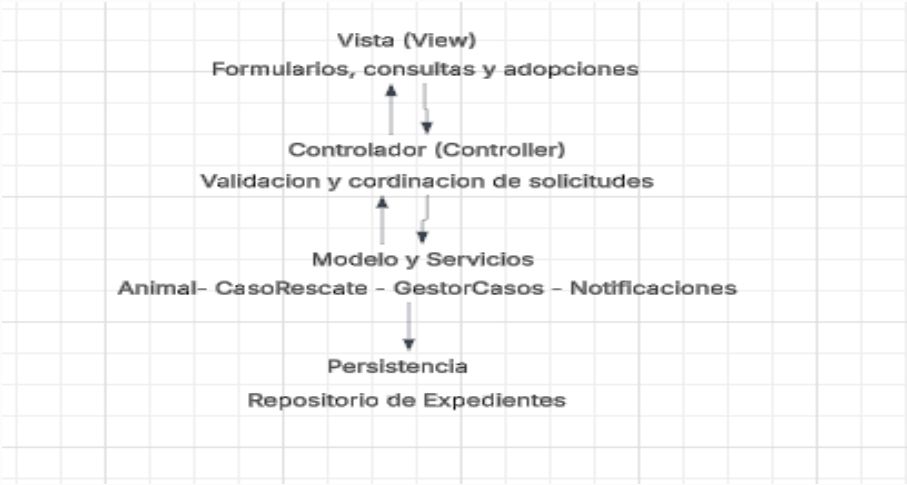
- **Capa de persistencia:** proporciona las operaciones necesarias para guardar y consultar los expedientes.

Funcionamiento de la arquitectura

Cuando un administrador registra un rescate, la vista envía los datos al controlador. El controlador verifica que la información requerida sea válida y solicita al servicio correspondiente la creación del expediente.

La fábrica crea el objeto que representa la especie seleccionada. Después, el gestor de casos coordina el registro en el repositorio y solicita al servicio de notificaciones que comunique el nuevo rescate a los voluntarios suscritos.

Finalmente, el sistema informa al usuario si el registro se realizó correctamente o si debe corregir algún dato.



6.3. Relación con los patrones de diseño

La arquitectura integra los patrones seleccionados de la siguiente manera:

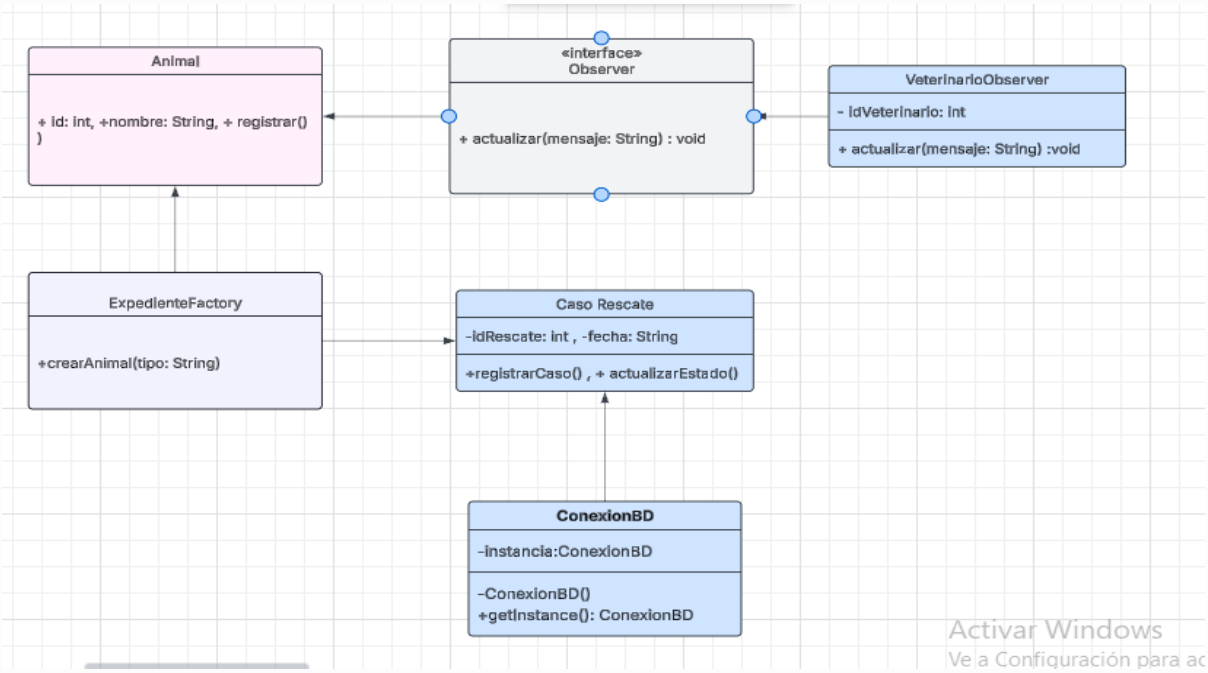
- Singleton se utiliza en el gestor central de casos.
- Factory Method separa la creación de objetos según la especie.
- Observer permite notificar eventos a los componentes suscritos.
- MVC organiza la interacción entre la vista, el controlador y el modelo.

Esta distribución pretende reducir el acoplamiento entre componentes y facilitar la ampliación de las funcionalidades. La implementación deberá verificar que las responsabilidades y las dependencias se mantengan de acuerdo con el diseño.

7. Modelado UML

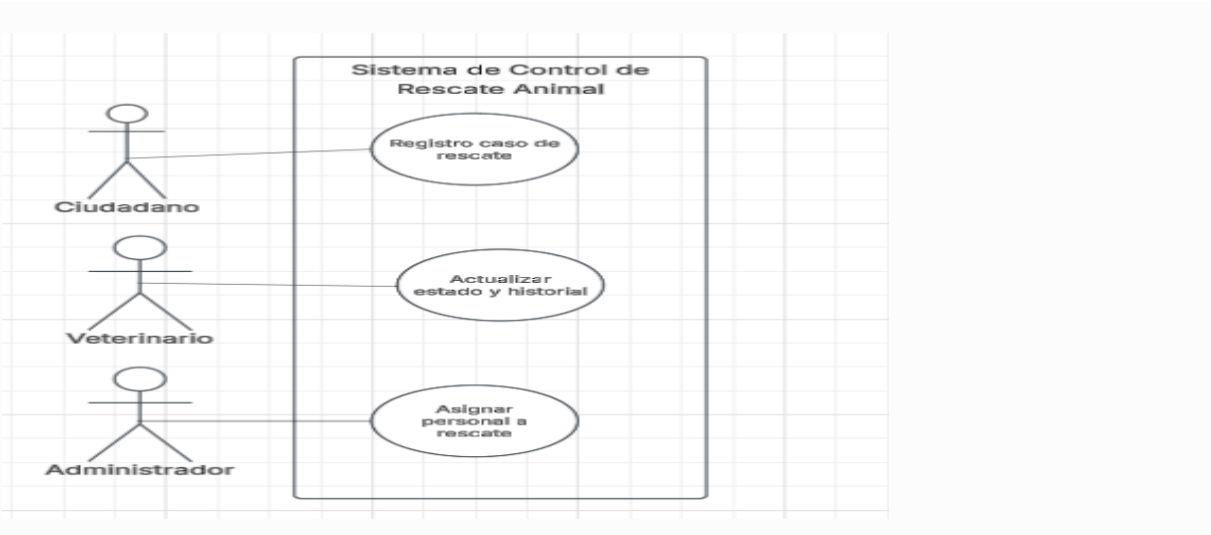
El Lenguaje Unificado de Modelado (UML) permite representar gráficamente la estructura y el comportamiento de un sistema antes de implementarlo. Para esta propuesta se utilizarán tres diagramas: casos de uso, clases y secuencia.

7.1 Diagrama de casos de uso



El ciudadano puede reportar animales que necesitan ayuda y consultar la información permitida de su reporte. El administrador registra los rescates, clasifica las especies, asigna responsables y gestiona las adopciones. El voluntario consulta sus casos asignados y actualiza la información operativa autorizada. El veterinario registra los tratamientos y las observaciones médicas.

La relación include indica que la clasificación de la especie forma parte del proceso de registrar un rescate.



8. Patrones de diseño aplicados

Los patrones de diseño ofrecen soluciones reutilizables a problemas comunes de organización del software. En esta propuesta se seleccionaron cuatro patrones que cumplen funciones diferentes y complementarias.

8.1. Singleton

Creacional: Singleton permite controlar la creación de una única instancia de una clase dentro de un proceso de la aplicación. En este proyecto se propone utilizarlo en GestorCasos, que coordina las operaciones principales de los expedientes.

Su utilidad consiste en proporcionar un punto de acceso compartido al gestor y evitar la creación accidental de varios gestores independientes. Sin embargo, Singleton por sí solo no garantiza una instancia única entre varios servidores o procesos.

8.2. Observer

Comportamiento: Observer establece una relación entre un objeto que genera eventos y los componentes interesados en recibirlos. En el sistema propuesto, ServicioNotificaciones informará a los voluntarios suscritos cuando se registre un nuevo rescate.

Este patrón evita que el registro de un caso tenga que conocer directamente a cada destinatario. Además, permite incorporar otros observadores, como un panel de alertas o un servicio de correo electrónico, sin cambiar innecesariamente el proceso principal.

8.3. Factory Method

Creacional: Factory Method separa la creación de objetos del código que los utiliza. En el sistema, FabricaExpedientes permitirá crear objetos de tipo Perro, Gato o FaunaSilvestre según la especie seleccionada.

Su aplicación facilita mantener la lógica de creación en un componente definido y preparar el diseño para incorporar nuevas categorías de animales.

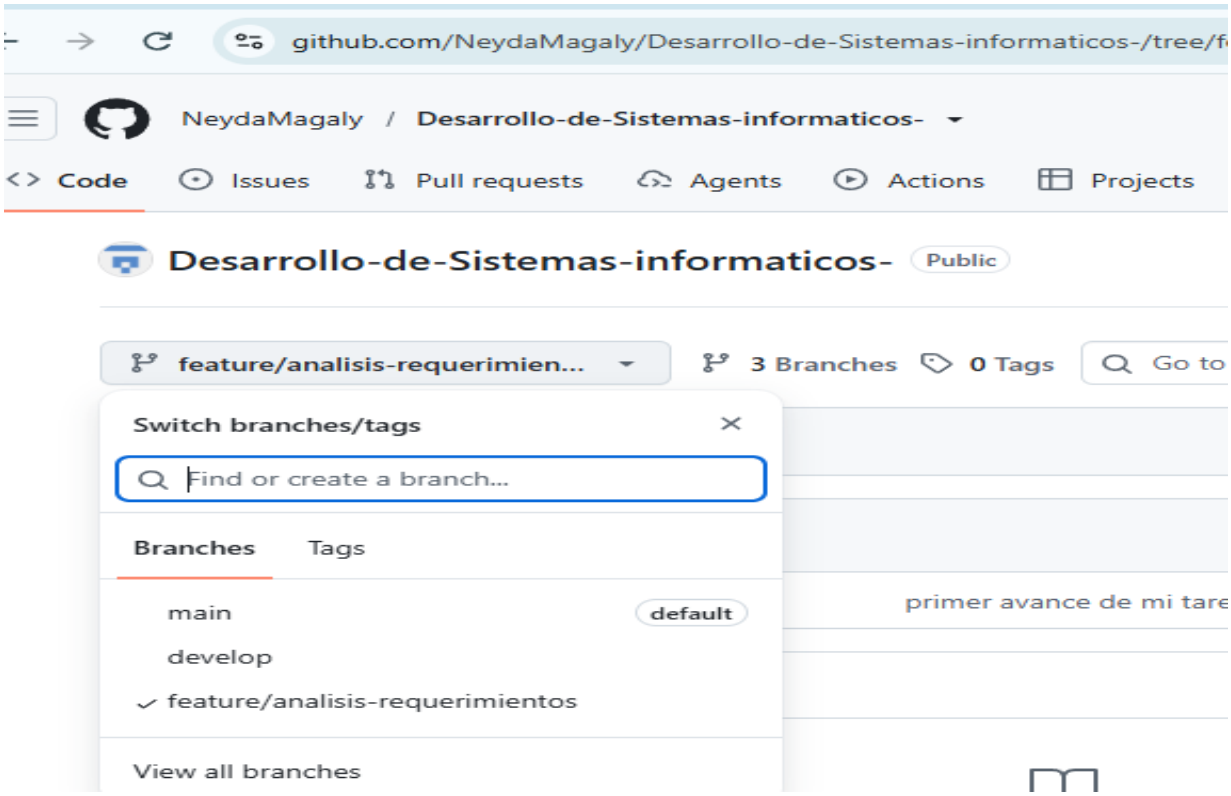
8.4. MVC (Model-View-Controller)

Tipo: patrón arquitectónico.

MVC separa la presentación, el procesamiento de solicitudes y el modelo del dominio. La vista muestra la información, el controlador recibe las acciones del usuario y el modelo representa las entidades y reglas de negocio.

En este proyecto, MVC permitirá modificar la interfaz sin alterar innecesariamente las reglas que controlan los expedientes. También facilitará probar los servicios de negocio de forma independiente.

Singleton	Creación accidental de varios gestores dentro de un mismo proceso	Acceso centralizado al gestor de casos
Observer	Dependencia directa entre el registro y cada destinatario de notificaciones	Menor acoplamiento y facilidad para agregar observadores
Factory Method	Lógica de creación de especies dispersa por el código	Creación organizada y ampliable
MVC	Mezcla de interfaz y reglas de negocio	Separación de responsabilidades y mayor mantenibilidad



9. Aplicación de los principios SOLID

Los principios SOLID ayudan a diseñar clases con responsabilidades claras y dependencias controladas. Su aplicación puede facilitar el mantenimiento y la ampliación del sistema de rescate y adopción animal.

9.1. Responsabilidad única (SRP)

Cada clase debe tener una responsabilidad principal. Por ejemplo, FabricaExpedientes se encarga de crear los objetos según la especie, mientras que ServicioNotificaciones se ocupa de comunicar los nuevos rescates.

9.2. Abierto/cerrado (OCP)

El sistema debe facilitar la incorporación de nuevas especies sin modificar innecesariamente las clases existentes. La abstracción Animal permite añadir nuevas clases derivadas, siempre que se respete el comportamiento definido.

9.3. Sustitución de Liskov (LSP)

Las clases Perro, Gato y FaunaSilvestre deben poder utilizarse donde se espere un objeto Animal, respetando las operaciones y condiciones establecidas por la clase base.

9.4. Segregación de interfaces (ISP)

Las interfaces deben contener operaciones relacionadas con un propósito concreto. Por ejemplo, ObservadorRescate solo necesita definir la operación de notificación y no debe obligar a implementar funciones de almacenamiento.

9.5. Inversión de dependencias (DIP)

Los componentes principales deben depender de abstracciones y no de implementaciones concretas. Por ello, GestorCasos utilizará la interfaz RepositorioCasos en lugar de depender directamente de una base de datos específica.

En conjunto, estos principios ayudan a reducir el acoplamiento y facilitan la evolución de la arquitectura.

10. Conclusión

El diseño propuesto ayuda a entender cómo un sistema informático puede ayudar a organizar las actividades de un refugio animal. A través del registro de casos, la clasificación de especies, la asignación de responsables y el seguimiento del estado de cada caso, se busca facilitar la gestión de la información y evitar que los expedientes se manejen de forma desordenada.

Los patrones de diseño elegidos cumplen funciones distintas dentro de la arquitectura.

Singleton centraliza el acceso al gestor de casos, Observar facilita las notificaciones, Factory Method organiza la creación de objetos según la especie y MVC separa la interfaz del sistema de su lógica. Para usarlos correctamente, es necesario aplicar una buena distribución de responsabilidades y seguir los principios SOLID.

Por último, GitHub y Gitflow permiten mantener un historial claro del trabajo y organizar las modificaciones antes de incorporarlas a la versión estable.

Con estos elementos se establece una base de diseño que puede servir para una futura implementación del sistema de rescate y adopción animal.

11. Referencias bibliográficas

Git. (s. f.). *Documentación de Git en español*. https://git-scm.com/docs/git/es.html

GitHub. (s. f.). *Documentación de GitHub en español*. https://docs.github.com/es

Gómez Seoane, C., Mayol Sarroca, E., Olivé Ramon, A., & Teniente López, E. (2003). *Diseño de sistemas software en UML*. Edicions UPC. https://hdl.handle.net/2099.3/36755

Larman, C. (2003). *UML y patrones: Introducción al análisis y diseño orientado a objetos y al proceso unificado* (2.ª ed.). Pearson Educación.

Object Management Group. (s. f.). *Lenguaje unificado de modelado (UML)*. https://www.omg.org/uml/

PlantUML. (s. f.). *Guía de referencia del lenguaje PlantUML*. https://plantuml.com/es/

12. Anexos y evidencias

